



SISTEMA DE GESTIÓN DE CITAS ITV

Nicolás Ortega Fernández 1º DAW

ÍNDICE

1. INTRODUCCIÓN

- 1.1. OBJETO DEL PROYECTO
- 1.2. ALCANCE
- 1.3. TECNOLOGÍAS UTILIZADAS

2. ANÁLISIS DE REQUISITOS

- 2.1. REQUISITOS FUNCIONALES
- 2.2. REQUISITOS NO FUNCIONALES
- 2.3. REQUISITOS DE INFORMACIÓN
- 2.4. REGLAS DE NEGOCIO

3. DISEÑO DEL SISTEMA

- 3.1. DIAGRAMA DE CASOS DE USO
- 3.2. DIAGRAMA DE ARQUITECTURA
- 3.3. DISEÑO DE LA BASE DE DATOS
- 3.4. DIAGRAMAS DE SECUENCIA

4. IMPLEMENTACIÓN TÉCNICA

5. CALIDAD Y PRUEBAS (QA)

- 5.1. PLAN DE PRUEBAS UNITARIAS
- 5.2. FLUJO DE TRABAJO Y CONTROL DE VERSIONES

6. ANÁLISIS ECONÓMICO

7. PRESENTACIÓN YOUTUBE

1. INTRODUCCIÓN

1.1. OBJETO DEL PROYECTO

Este proyecto tiene como objetivo principal el desarrollo de una aplicación de escritorio denominada **ITV-Manager**, destinada a la gestión de citas de vehículos para la estación Luis Vives.

El sistema busca centralizar el registro de citas, la validación de datos y el cumplimiento de las restricciones de seguridad utilizando para ello tecnologías .NET (C# 14 y .NET 10).

El sistema está construido de manera limpia y respetando los principios SOLID, garantizando así una persistencia polimórfica que permite alternar entre 3 motores de bases de datos (ADO.NET, Dapper y Entity Framework) sin afectar en nada a la interfaz ni al usuario.

1.2. ALCANCE

En este apartado se definen las funcionalidades del sistema. Estas son:

- **Gestión de Citas.** Registro completo de inspecciones vinculadas a un vehículo (matrícula, marca, modelo y fecha de matriculación) y un propietario (DNI).
- **Motor de Reglas de Negocio.** Implementación de validaciones en tiempo real. Esto incluye DNI válido, matrícula española y restricciones en las fechas.
- **Triple Capa de Persistencia.** Hasta tres módulos de acceso a datos (ADO.NET, Dapper y Entity Framework Core) intercambiables y configurables mediante un archivo externo JSON.
- **Buscador Avanzado.** Sistema de filtrado multiparamétrico con paginación de resultados para optimizar la UI.
- **Módulo de Salida de Datos.** Funcionalidad para exportar registros en JSON, XML, CSV, además de generación de fichas de cita en HTML y PDF.
- **Sistema de Trazabilidad (Logs).** Registro de eventos del sistema en archivos físicos.

1.3. TECNOLOGÍAS UTILIZADAS

Para el desarrollo de ITV-Manager se ha seleccionado un conjunto de tecnologías que garantizan un rendimiento óptimo:

A. Stack Tecnológico

- Lenguaje de Programación: C# 14, aprovechando las últimas mejoras en sintaxis.
- Framework: .NET 10, la versión más reciente de Microsoft.
- Interfaz de Usuario: WPF mediante lenguaje de marcas XAML, permitiendo una separación clara entre la lógica de negocio y la vista (patrón MVVM).
- Motores de Persistencia:
 - o ADO.NET: control granular y de bajo nivel.
 - o Dapper: mapeo ágil de objetos.
 - o Entity Framework Core: ORM completo para gestión de entidades.

B. Herramientas de Desarrollo

- Entorno de Desarrollo Integrado: Rider, seleccionado por su avanzado motor de análisis de código y soporte para refactorización. Se usa Visual Studio Code como visor rápido de código.
- Diseño de Interfaz: XAML.io, utilizado para prototipado rápido de componentes visuales.
- Gestión de Versiones: Git bajo metodología GitFlow, con alojamiento en repositorio de GitHub.
- Documentación: Word para la redacción y Canva para la portada.
- Bases de Datos: SQLite.

2. ANÁLISIS DE REQUISITOS

2.1 REQUISITOS FUNCIONALES

- RF-01: Alta de Cita. El sistema permite registrar una nueva cita siguiendo las reglas de negocio.
- RF-02: Edición de Cita. El administrador puede modificar cualquier dato de una cita. El sistema siempre revalida todas las reglas de negocio tras la modificación.
- RF-03: Baja de Cita. El sistema permite eliminar una cita. El tipo de borrado (físico o lógico) dependerá de la configuración externa.
- RF-04: Consulta de Citas. El sistema muestra un listado de todas las citas almacenadas de forma paginada, de 10 en 10.
- RF-05: Validación de Reglas de Negocio. El sistema asegura que la matrícula sea con el formato español actual, el DNI sea válido para el Ministerio del Interior, la fecha de matriculación sea posterior a la fecha actual o a la fecha de la cita y que la fecha de inspección esté dentro de los próximos 30 días. Además, valida que una matrícula no tenga 2 citas el mismo día y que un DNI no tenga más de 3 vehículos asociados para la misma fecha.

- RF-06: Búsqueda Multiparámetro. El sistema permite filtrar las citas por matrícula, DNI, marca, modelo o tipo de motor simultáneamente.
- RF-07: Importación de Datos. El sistema permite insertar citas desde archivos JSON, CSV y XML.
- RF-08: Exportación de Datos. El sistema permite exportar las citas a archivos JSON, CSV y XML.
- RF-09: Generación Comprobantes. El sistema permite imprimir la ficha de la cita en HTML y PDF.
- RF-10: Restaurar Cita. El sistema permite restaurar una cita eliminada lógicamente

2.2. REQUISITOS NO FUNCIONALES

- RNF-01: Persistencia Polimórfica: la aplicación es capaz de cambiar su motor de datos (ADO.NET, Dapper o Entity Framework) basándose únicamente en un archivo de configuración.
- RNF-02: Trazabilidad (Logging): todos los eventos críticos y errores están registrados en un log (archivo).
- RNF-03: Feedback Visual: la interfaz WPF reacciona a los errores de validación en tiempo real (por ejemplo, marcando en rojo un campo incorrecto) antes de intentar guardar los datos.
- RNF-04: Independencia de Compilación: cambios críticos como el motor de persistencia o el tipo de borrado (lógico/físico) no requieren que el programador vuelva a compilar el código ya que estos se leen de un archivo JSON al arrancar.
- RNF-05: Compatibilidad de Datos: el sistema garantiza la integridad de los datos independientemente de si se exportan a JSON, XML o CSV.

2.3. REQUISITOS DE INFORMACIÓN

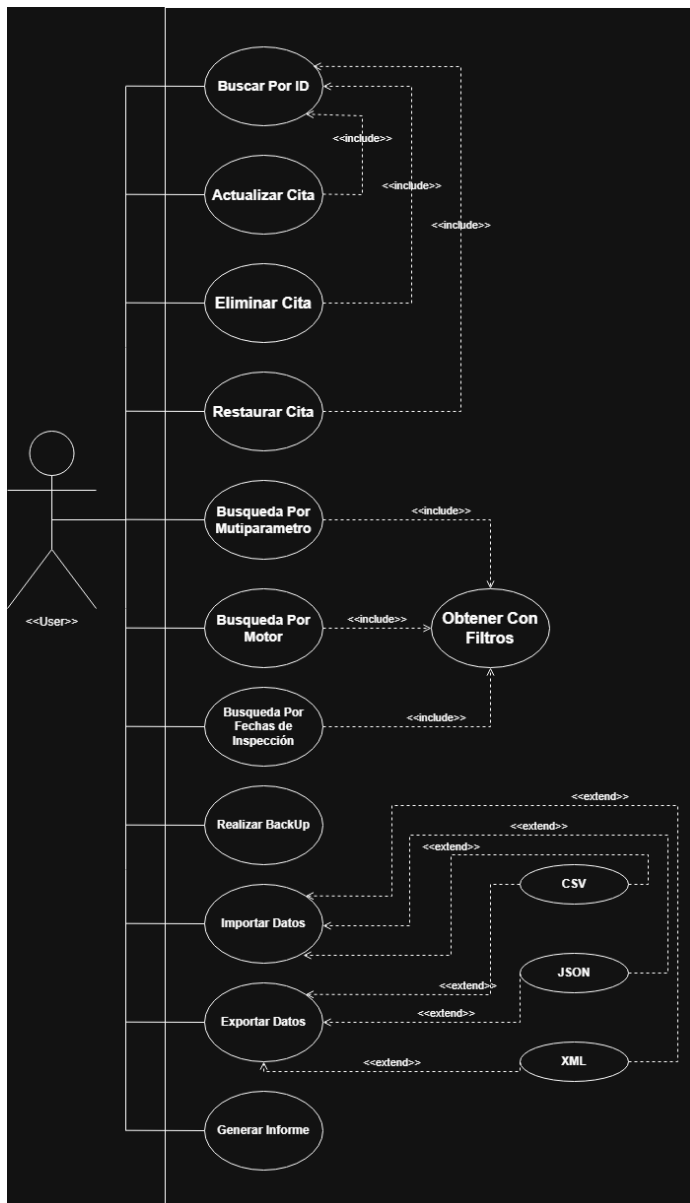
- RI-01: Id Cita. Entero.
- RI-02: Matrícula. Texto.
- RI-03: Marca. Texto.
- RI-04: Modelo. Texto.
- RI-05: Tipo de Motor. Enum (Gasolina, Diésel, Eléctrico, Híbrido).
- RI-06: Fecha de Matriculación. Fecha.
- RI-07: DNI. Texto.
- RI-08: Fecha Inspección. Fecha.
- RI-09: Motor de Datos Activo. (ADO.NET / Dapper / EF Core).
- RI-10: Modo de Borrado. (Físico / Lógico).
- RI-11: Configuración de Logs. Ruta de almacenamiento.

2.4. REGLAS DE NEGOCIO

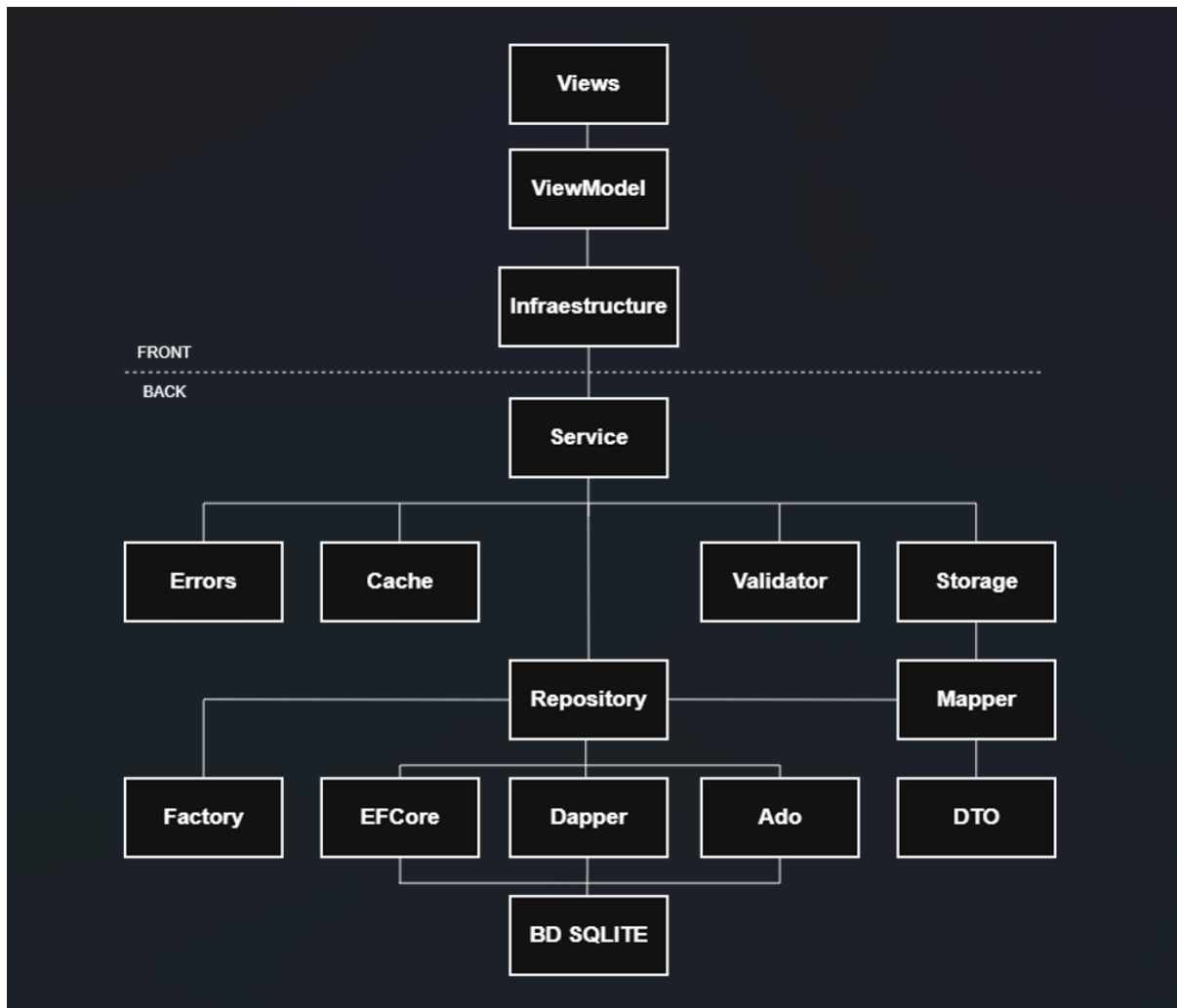
- RN-01: Integridad de la Matrícula. El sistema rechaza cualquier matrícula que no cumpla con el formato actual español: cuatro dígitos seguidos de tres letras consonantes (NNNNLLL).
- RN-02: Integridad del DNI. El sistema rechaza cualquier dni que no cumpla con el formato NNNNNNNNL o que la letra no corresponda al número.
- RN-03: Coherencia de Fecha de Matriculación. El sistema sólo acepta fechas de matriculación anteriores a la actual.
- RN-04: Ventana Temporal de Inspección. Las citas de inspección solo pueden programarse en un intervalo de 30 días desde la fecha actual.
- RN-05: Única Inspección Diaria. Solo se permite una cita por coche al día.
- RN-06: Cupo Máximo por Propietario. Un mismo DNI sólo puede tener 3 citas en un día.
- RN-07: Persistencia Transparente. El sistema puede alternar su motor de base de datos (ADO.NET, Dapper o EF Core) mediante un archivo de configuración externo.
- RN-08: Política de Borrado. El administrador decide, vía configuración, si la eliminación de un registro es un borrado físico o un borrado lógico.

3. DISEÑO DEL SISTEMA

3.1 DIAGRAMA DE CASOS DE USO



3.2 DIAGRAMA DE ARQUITECTURA



3.3 DISEÑO DE LA BASE DE DATOS

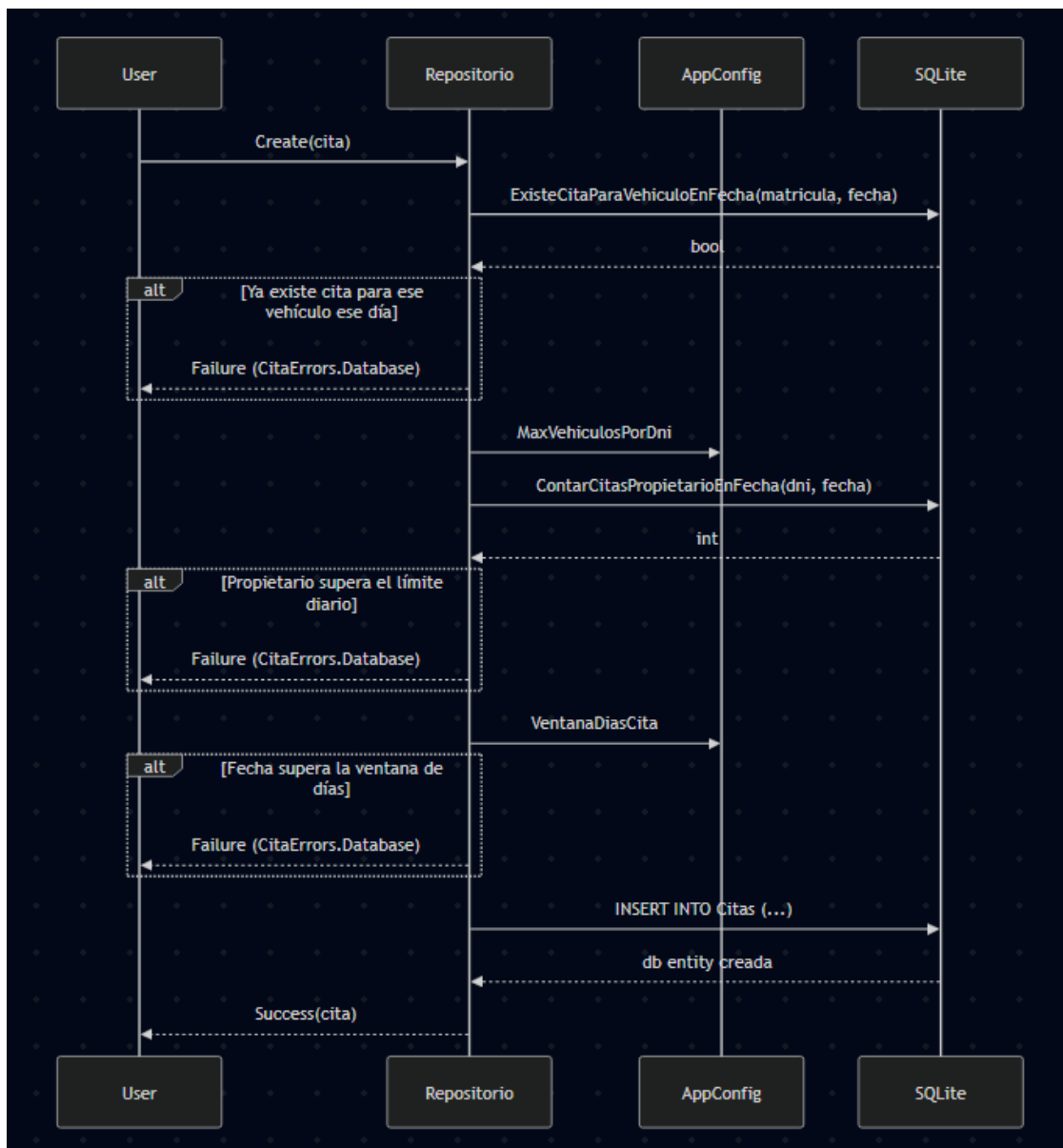
CREATE TABLE IF NOT EXISTS Citas (
Id INTEGER PRIMARY KEY,
Matricula TEXT NOT NULL,
Marca TEXT NOT NULL,
Modelo TEXT NOT NULL,
Cilindrada INTEGER NOT NULL,
Motor INTEGER NOT NULL,
Dni TEXT NOT NULL,
FechaMatriculacion TEXT NOT NULL,
FechaInspeccion TEXT NOT NULL,
CreatedAt TEXT NOT NULL,

UpdatedAt TEXT NOT NULL,
IsDeleted INTEGER NOT NULL DEFAULT 0,
DeletedAt TEXT

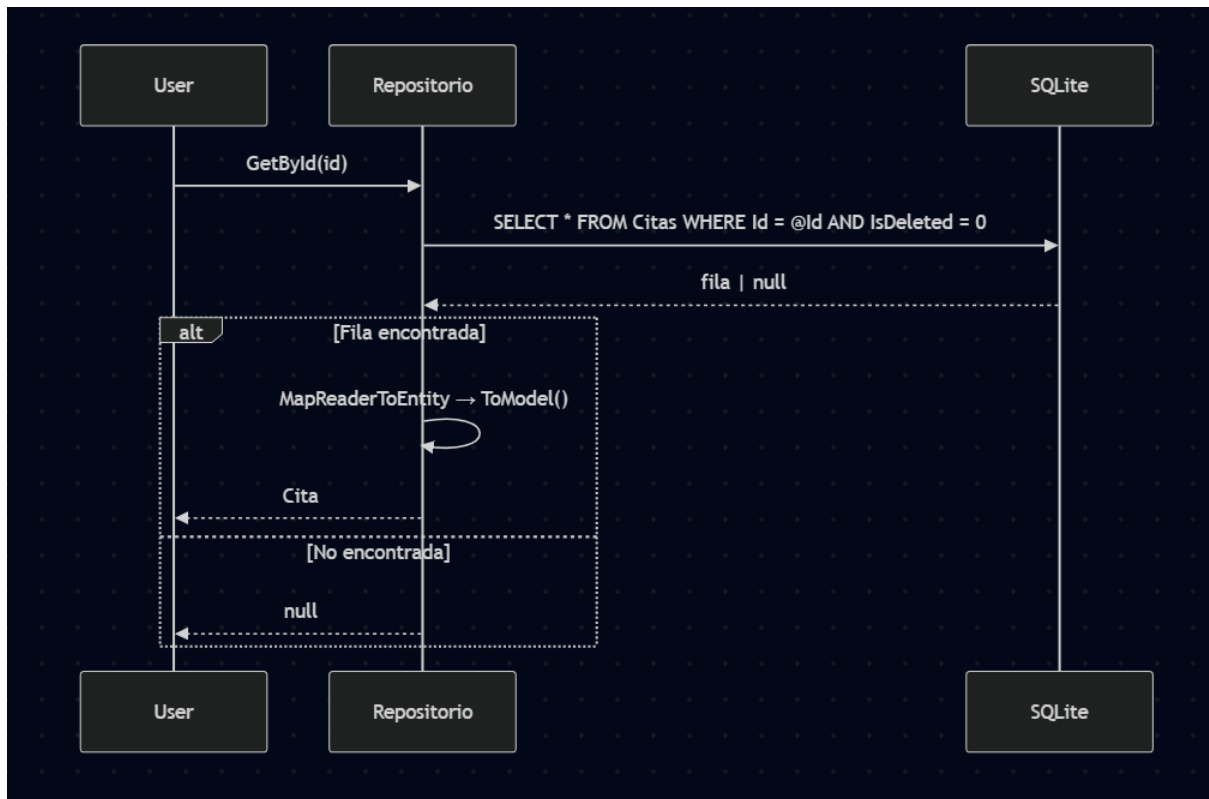
)

3.4 DIAGRAMAS DE SECUENCIA

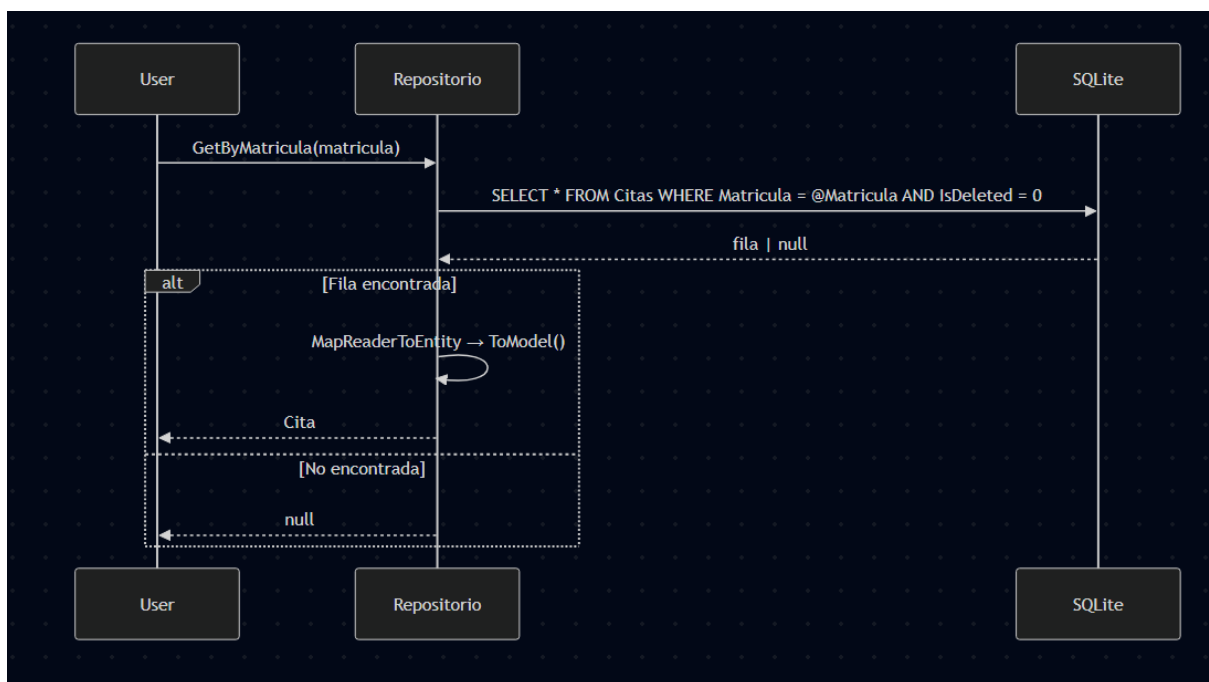
CREATE



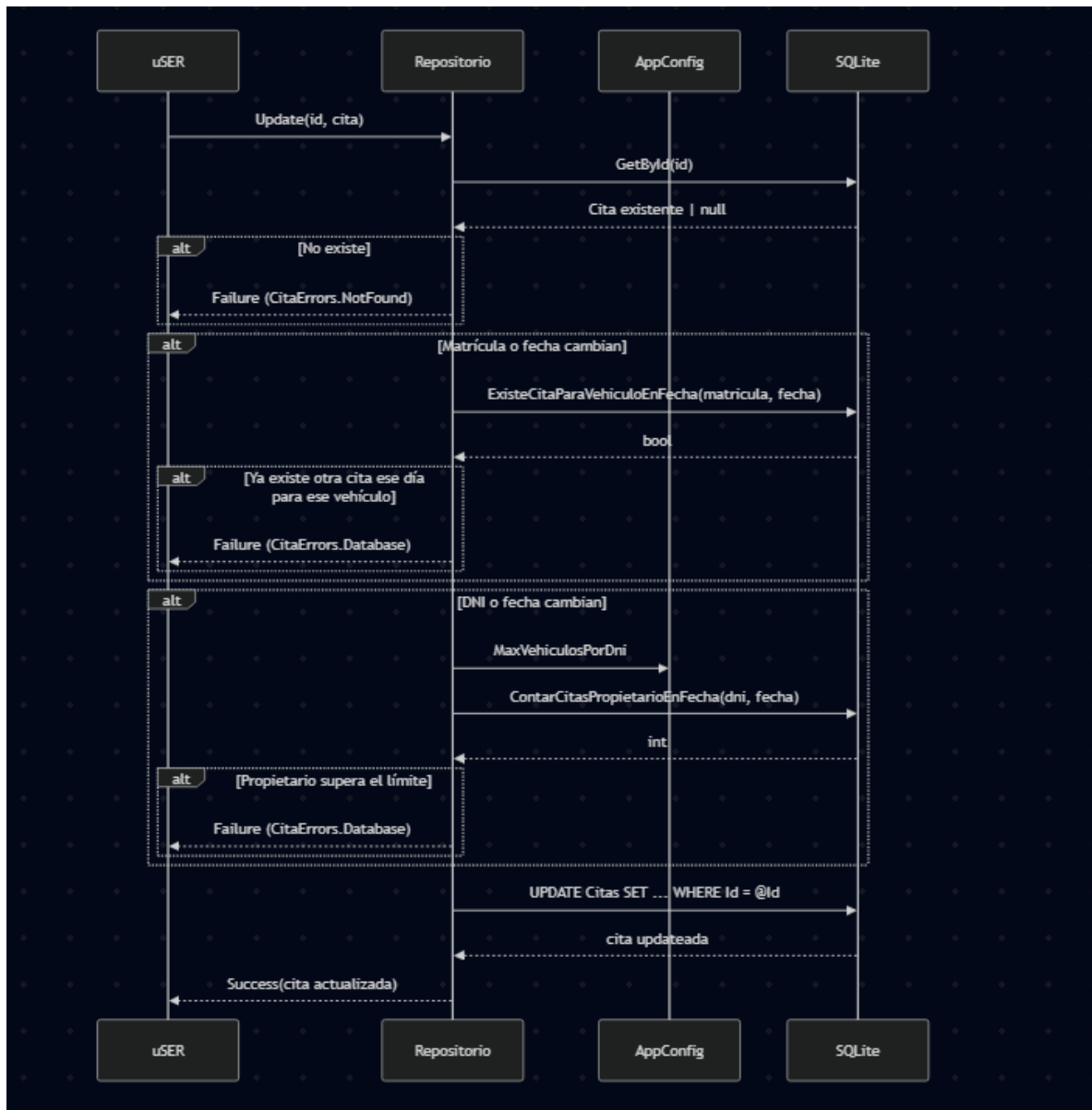
GET BY ID



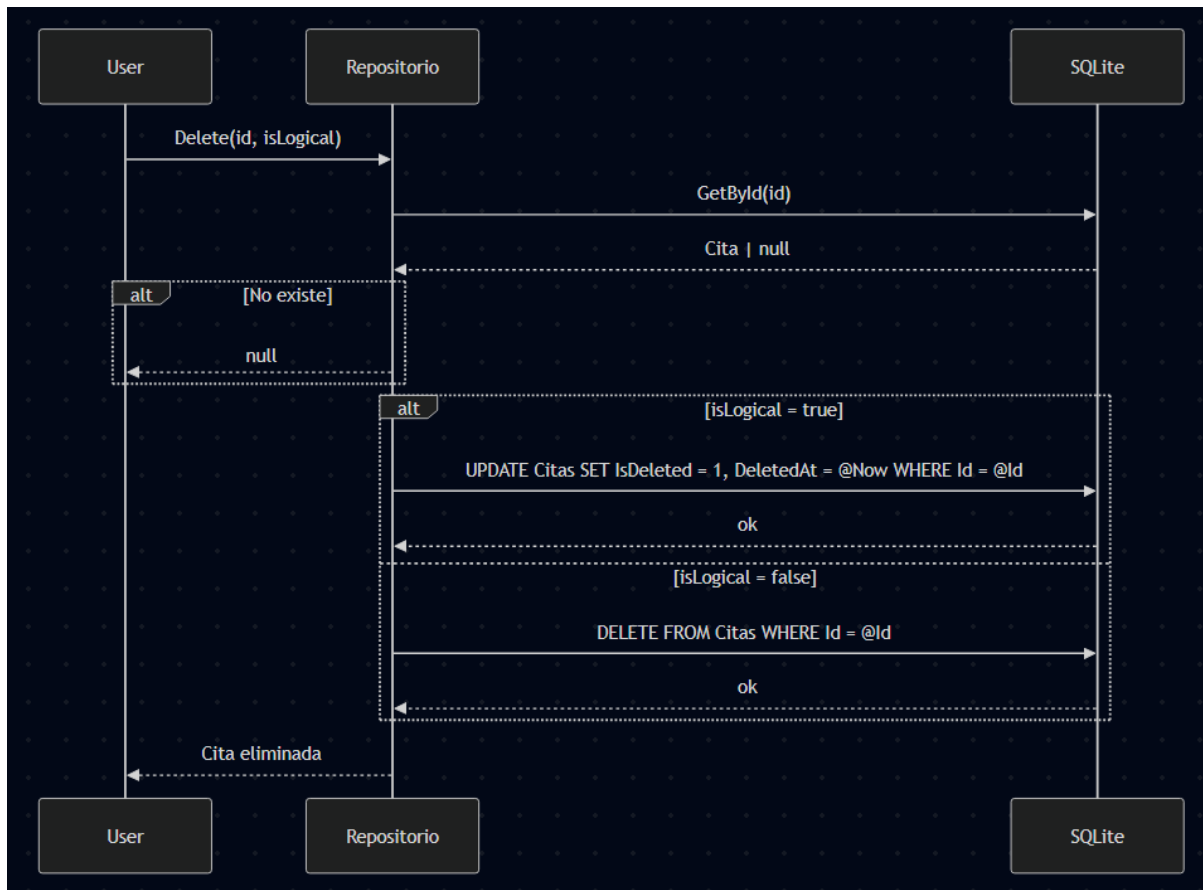
GET BY MATRÍCULA



UPDATE



DELETE



4. IMPLEMENTACIÓN TÉCNICA

La aplicación ha sido desarrollada utilizando una arquitectura en capas basada en los principios SOLID. Estas son algunas de las herramientas usadas y su justificación:

- **WPF:** se usó WPF como tecnología de interfaz debido a que el proyecto requería una aplicación de escritorio con interacción de usuario y manejo intensivo de datos. Las principales ventajas de usar WPF en este proyecto son la integración directa con .NET y la separación clara entre interfaz y lógica mediante MVVM, además de ventajas como las validaciones visuales.
- **MVVM:** se ha usado el patron Modelo Vista Vista Modelo debido a que es el mejor para aplicaciones WPF. Se ha preferido por encima del patrón MVC ya que este tiene peor implementación con XAML, además de que este no aprovecha el DataBinding.
- **RESULT:** se ha usado la ROP para la gestión de errores y validaciones. Se ha preferido el uso de la ROP por encima de las excepciones ya que estas, como su nombre indica, estan pensadas para casos excepcionales, y que una matrícula contenga una vocal no es un caso excepcional, es un error de validación que puede ser corregido. Además, gracias a usar los Result, el código es mucho más limpio y los errores son más predecibles.

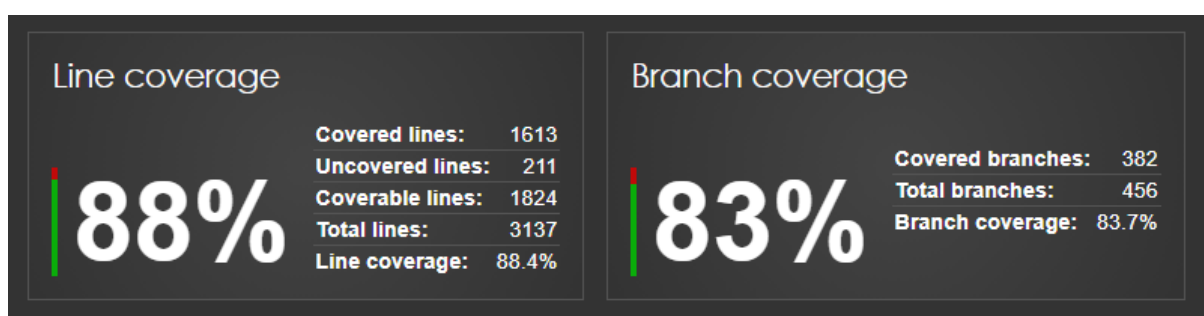
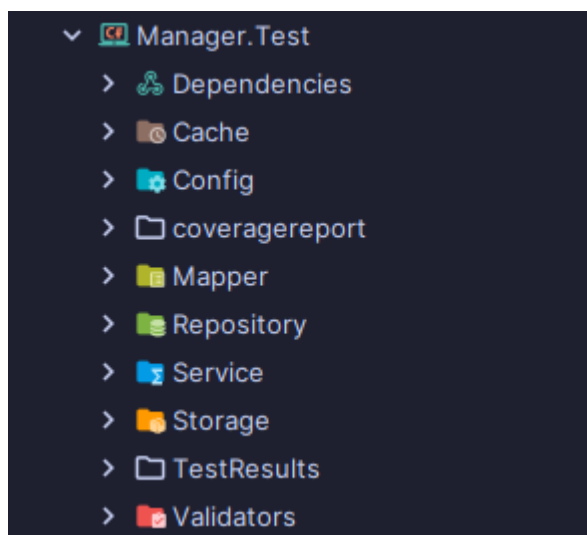
- **INYECCIÓN DE DEPENDENCIAS:** se ha usado principalmente para facilitar los test y para cumplir el requisito del proyecto de poder cambiar el motos de base de datos desde un archivo .json.

5. CALIDAD Y PRUEBAS (QA)

5.1. PLAN DE PRUEBAS UNITARIAS

Se han llevado a cabo pruebas unitarias y pruebas de dobles con la finalidad de verificar y garantizar la calidad del código.

Estas pruebas han ayudado a verificar que el código programado hace lo esperado



5.2. FLUJO DE TRABAJO Y CONTROL DE VERSIONES

Se ha usado Git con repositorio alojado en Github durante todo el desarrollo para tanto poder trabajar desde varios dispositivos como para llevar a cabo un control de los cambios en el proyecto y ver cómo ha ido evolucionando.

Se adjunta a continuacion el diagrama de GitFlow:

Text or hash	(*) Aa	Branch	User	Date	Paths	
merge to main		origin & main	Nicolás Ortega Fernández*	5 minutes ago		
end code		origin/dev	nicolasorteg	1 hour ago		
fix logs			nicolasorteg	Yesterday 20:57		
inicio implement restore			nicolasorteg	Yesterday 17:21		
fix tests			nicolasorteg	Yesterday 5:31		
implementado busqueda por id y matricula (cambios getid repo)			nicolasorteg	Yesterday 4:56		
implementado create cita			nicolasorteg	Yesterday 4:23		
implementado update citas			nicolasorteg	Yesterday 4:10		
add backup			nicolasorteg	Yesterday 3:45		
conecta storage para import y export			nicolasorteg	Yesterday 2:29		
avanza viewmodels + update en repositories			nicolasorteg	25/05/2026 22:25		
pequeños fixes			nicolasorteg	25/05/2026 18:49		
fix test			nicolasorteg	25/05/2026 18:14		
init front			nicolasorteg	25/05/2026 16:27		
end back			nicolasorteg	25/05/2026 5:27		
fix modelo			nicolasorteg	25/05/2026 4:57		
end service			nicolasorteg	25/05/2026 4:30		
add test cache			nicolasorteg	25/05/2026 3:51		
end manager service			nicolasorteg	25/05/2026 3:45		
add clear cache			nicolasorteg	25/05/2026 3:42		
elimina error			nicolasorteg	25/05/2026 2:53		
end html y pdf			nicolasorteg	24/05/2026 23:37		
end report service test			nicolasorteg	24/05/2026 23:02		
end reports service			nicolasorteg	24/05/2026 22:43		
end test			nicolasorteg	24/05/2026 20:07		
end service storage			nicolasorteg	24/05/2026 19:48		
test ado			nicolasorteg	24/05/2026 19:12		
appconfig test			nicolasorteg	23/05/2026 19:41		
end error test			nicolasorteg	23/05/2026 19:20		
end test repos			nicolasorteg	23/05/2026 19:07		
add blindaje RN en restores			nicolasorteg	23/05/2026 18:47		
end test repo, falta testear un ajuste aun no implementado			nicolasorteg	23/05/2026 18:40		
refactor storage construtores			nicolasorteg	23/05/2026 17:09		

● end dapper test	nicolasorteg	23/05/2026 4:22
● end cache test	nicolasorteg	22/05/2026 16:54
● end cache	nicolasorteg	22/05/2026 13:39
● add ICache	nicolasorteg	22/05/2026 13:23
● add inyector de dependencias	nicolasorteg	22/05/2026 5:14
● create mensaje de update para las vistas	nicolasorteg	22/05/2026 5:06
● test	nicolasorteg	22/05/2026 5:00
● end dapper repo	nicolasorteg	22/05/2026 4:54
● init dapper rep	nicolasorteg	22/05/2026 4:14
● add getall + end efcore rep	nicolasorteg	22/05/2026 4:11
● avanza repo	nicolasorteg	21/05/2026 20:58
● init efcore rep	nicolasorteg	21/05/2026 20:35
● end database ado	nicolasorteg	21/05/2026 20:24
● end json test	nicolasorteg	21/05/2026 18:24
● end csv tests	nicolasorteg	21/05/2026 18:00
● setup csv	nicolasorteg	20/05/2026 20:51
● create csv test	nicolasorteg	20/05/2026 20:46
● add json storage	nicolasorteg	19/05/2026 14:32
● add json interface	nicolasorteg	19/05/2026 13:51
● end csv storage	nicolasorteg	19/05/2026 13:50
● deletedat to nullable	nicolasorteg	19/05/2026 13:36
● quick fix	nicolasorteg	19/05/2026 13:35
● add citacsvstorage	nicolasorteg	19/05/2026 12:44
● refactor comentarios + add ICitaCsvStorage	nicolasorteg	19/05/2026 12:37
● add IStorage	nicolasorteg	19/05/2026 12:34
● avanza ado repo	nicolasorteg	18/05/2026 21:35
● init ado.net repo	nicolasorteg	18/05/2026 18:47
● create cita factory	nicolasorteg	18/05/2026 18:06
● end ICitaRepository	nicolasorteg	18/05/2026 17:51
● end ICrudRepository	nicolasorteg	18/05/2026 17:37
● nameof test	nicolasorteg	18/05/2026 17:17
● end mapper test 100%	nicolasorteg	14/05/2026 20:47

fix mapper + refacto tests	nicolasorteg	14/05/2026 20:16
init mapper test	nicolasorteg	14/05/2026 19:42
refactor entity + end mapper	nicolasorteg	14/05/2026 18:28
fix	nicolasorteg	14/05/2026 18:05
end orm	nicolasorteg	14/05/2026 18:04
cita entity end	nicolasorteg	14/05/2026 17:52
cita dto end	nicolasorteg	14/05/2026 17:40
end validator	nicolasorteg	13/05/2026 20:53
quick refactor	nicolasorteg	13/05/2026 20:40
validator fixes, is letter	nicolasorteg	13/05/2026 20:31
casos corrector validator + fixes en en validator	nicolasorteg	13/05/2026 20:30
comienzo primeros tests validator	nicolasorteg	13/05/2026 18:45
add solucion tests	nicolasorteg	13/05/2026 18:21
add debug logger	nicolasorteg	13/05/2026 18:11
end validator	nicolasorteg	13/05/2026 16:38
fix matricula	nicolasorteg	13/05/2026 16:25
funciones de extension terminadas	nicolasorteg	13/05/2026 16:16
add cilindrada	nicolasorteg	12/05/2026 20:44
validador dni + marca + modelo extension	nicolasorteg	12/05/2026 20:36
validador matricula extension	nicolasorteg	12/05/2026 20:15
interfaz validador con result	nicolasorteg	12/05/2026 18:47
model cita end	nicolasorteg	12/05/2026 18:28
lectura appsettings.json mediante config	nicolasorteg	11/05/2026 18:21
end settings	nicolasorteg	08/05/2026 16:58
end storage errors	nicolasorteg	08/05/2026 16:43
errores de citas acabados	nicolasorteg	08/05/2026 16:14
add citas errores	nicolasorteg	08/05/2026 16:04
add domainerror	nicolasorteg	08/05/2026 15:45
add appsettings.json	nicolasorteg	06/05/2026 21:01
paquetes instalados	nicolasorteg	06/05/2026 20:52
init solucion	nicolasorteg	06/05/2026 20:34
add gitignore	nicolasorteg	06/05/2026 20:18
Initial commit	Nicolás Ortega Fernández*	30/04/2026 18:44

6. ANÁLISIS ECONÓMICO

La estimación de costes va a estar dividida 3 partes.

- Coste de producción.
- Coste de tecnologías.
- Coste de mantenimiento.

COSTE DE PRODUCCIÓN

La producción del sistema gestor ha constado de un aproximado de **114h**.

Han sido 2-3 semanas programando 6 días a la semana unas 5-6h diarias.

Si se pone de supuesto que un sueldo para desarrollador .NET junior es de 12€/h:

12 x 100 = **1368€ de costes de producción**

COSTE DE TECNOLOGÍAS

Las tecnologías han sido proporcionadas por el centro y por lo tanto no suponen un coste real.

Sin embargo, en el supuesto de tratarse de un desarrollo independiente sin proveedores de tecnologías, habría que añadir un aproximado de **150€** por la licencia del Rider de JetBrains. Este IDE es de uso gratuito con fines no comerciales, pero al tratarse de un proyecto comercial la licencia sería necesaria y obligatoria.

COSTE DE MANTENIMIENTO

En cuanto al mantenimiento se ha calculado aplicando un 25% al coste total de desarrollo:

$$1368 + 150 = 1518€$$

$$25\% \text{ de } 1518€ = 380€$$

Este mantenimiento incluye 2 años, por lo que se duplica el coste de mantenimiento y se queda en **740€** de mantenimiento.

COSTE TOTAL

Sumando los 3 tipos de coste se obtiene que este proyecto costaría un estimado de

2258€

7. PRESENTACIÓN YOUTUBE

Enlace -> <https://youtu.be/z0-WcZMReBk>